OPENCHAIN AUTOMOTIVE WORK GROUP

Automotive SBOM Framework v1.0

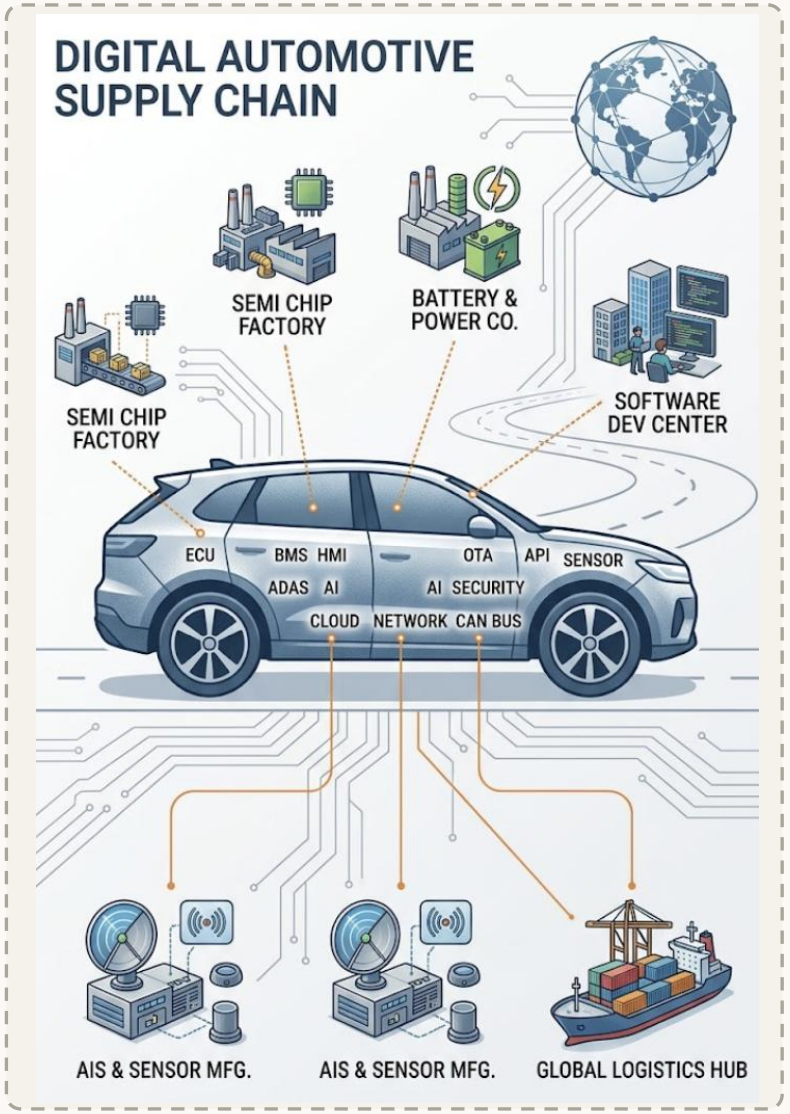
A shared best practice for automotive supply chains

Masato Endo · Automotive Chair and Board Member of OpenChain Project

Ayumi Watanabe · Hitachi Solutions, Ltd.

Mary Meixia Wang · Executive Director for OpenChain Project

Keisuke Takase · Toyota Motor Corporation



github.com/OpenChain-Project/Automotive-SBOM

Agenda and three things to take away

01 Background

02 Purpose and structure

03 Journey and members

04 Feedback and v1.0 changes

05 Toward v2.0

Three things to take away

1. **v1.0 is published**
   Released on GitHub in October 2026 after an open community review.

2. **A best practice, not a new standard**
   Built by the industry, on top of existing standards.

3. **v2.0 needs your input**
   VEX, functional safety, AI-era code, AIBOM, and proprietary software.

Why SBOMs matter for vehicles now

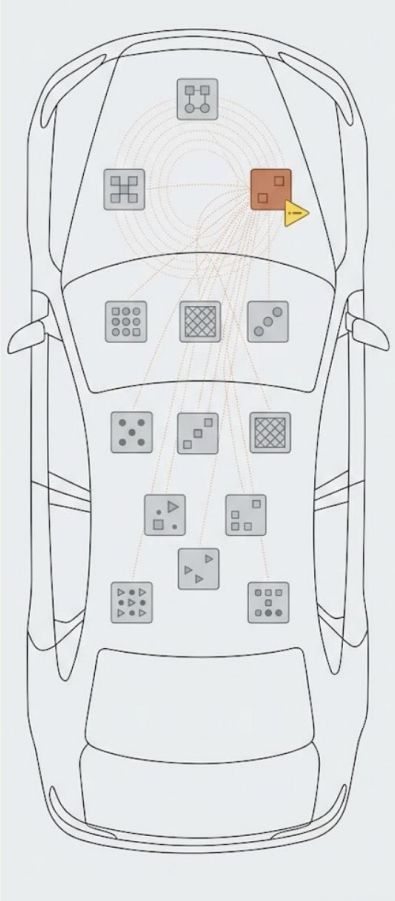
SBOM = a machine-readable list of the parts inside software

1. **Security**
   Find every affected ECU fast when a new CVE appears

2. **Compliance**
   UN R155 / R156, CISA and METI guidance, EU CRA *

3. **Supply chain**
   Software crosses many company borders to reach the car

* Vehicles mostly fall under sector rules, not the CRA itself.



Automotive SBOM

One supply chain, many SBOM "dialects"

Today

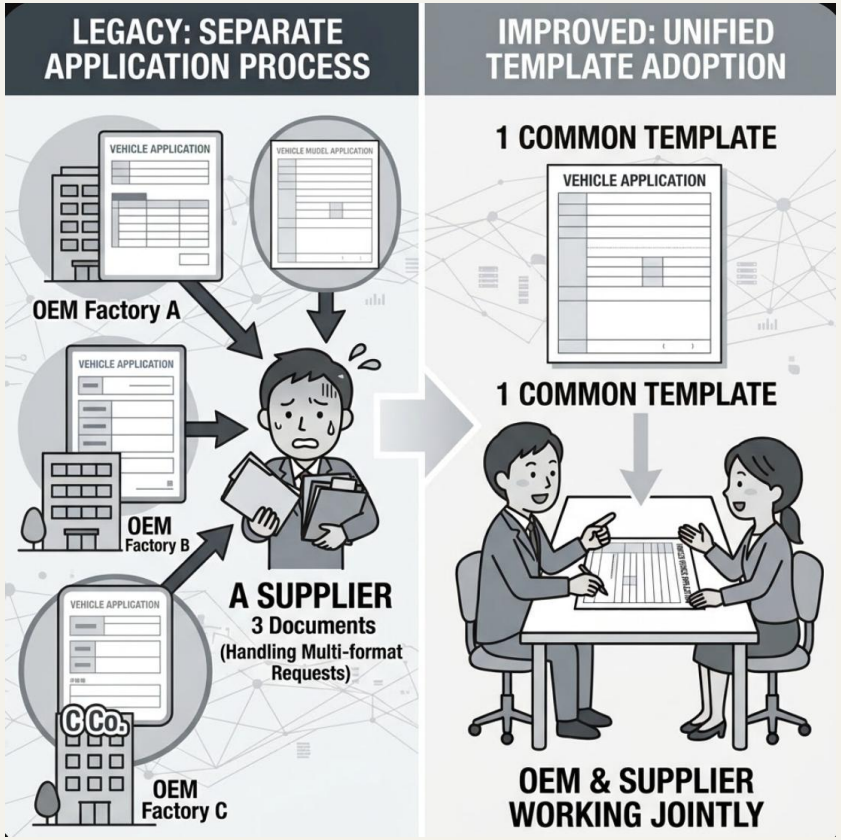
- Each OEM asks for different fields and granularity
- "Supplier" means different things to different teams
- Suppliers rebuild the same SBOM again and again

With a shared framework

- One set of fields, one meaning, any mainstream format
- Tools built against one set of requirements

Standards: NTIA 2021 · CISA · BSI TR-03183 · J-Auto-ISAC · 2026 Minimum Elements

Formats: SPDX 2.x · SPDX Lite · SPDX 3.x · CycloneDX 1.4–1.7



Automotive SBOM

What makes automotive different

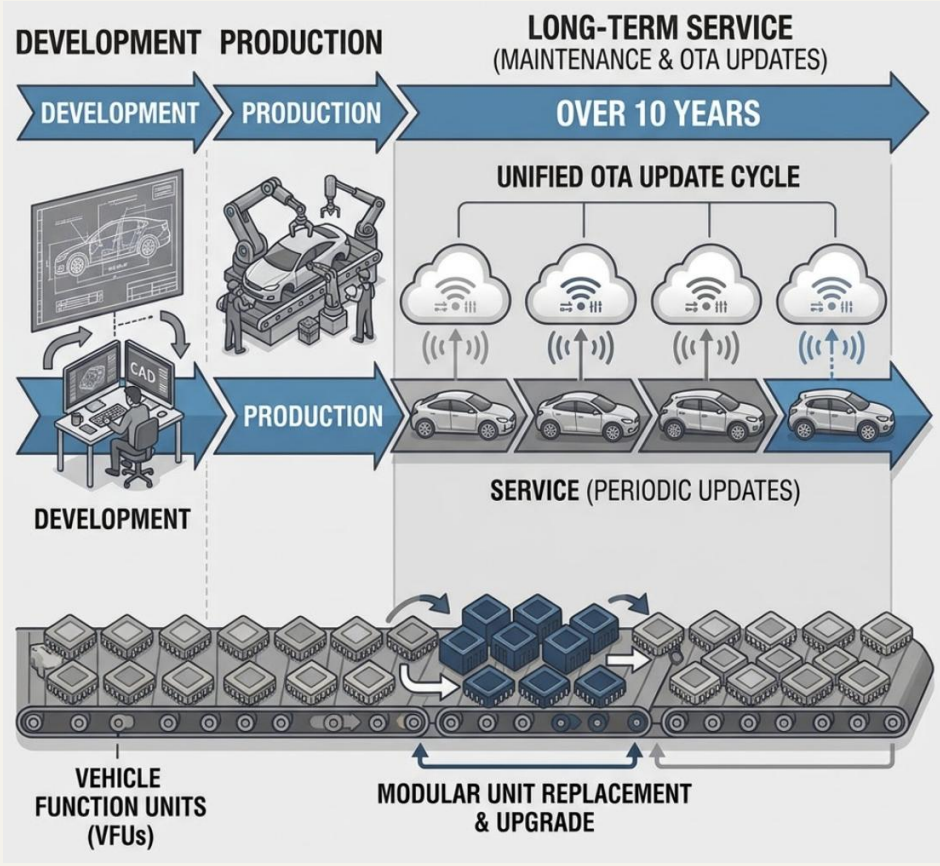
1. **Multi-tier supply chain**
   OEM ← Tier 1 ← Tier 2 ← … ← OSS and COTS

2. **Long lifecycle**
   10+ years in the field, updated over the air (OTA)

3. **Many ECUs**
   Each with its own software, versions, and variants

4. **Functional safety**
   A security fix is also a safety change → **v2.0**

5. **VEX and PSIRT**
   Vulnerability response spans many companies → **v2.0**



Our purpose: a best practice, not a new standard

What it is

- ✓ A best practice by the industry, for the industry
- ✓ Built on existing standards and guidance
- ✓ Independent of format, tool, and organization
- ✓ Open: public on GitHub under CC BY 4.0

What it is not

- ✗ A new SBOM standard or format
- ✗ A competitor to SPDX or CycloneDX
- ✗ A regulation or a certification scheme
- ✗ A way for one company to gain an edge

Collaborate where we don't compete — solve shared problems once, as one industry.

All meetings follow the Linux Foundation antitrust policy.

Automotive SBOM

Four objectives

01 Common language

Same component information and terms across the chain

02 Efficiency

Fewer company-specific requirements, less duplicated work

03 Risk management

Support vulnerability management and license compliance

04 Tool ecosystem

One clear set of requirements for SBOM tool providers

Intended for: OEMs · Tier-N suppliers · OSS compliance teams · PSIRTs · tool vendors · regulators

Automotive SBOM

Three layers of the framework

1

Data Fields

What to record

16 fields: 4 for the SBOM, 12 for each component

2

Automation Support

How to express it

Mapping to SPDX and CycloneDX, with examples

3

Practice and Process

How to operate it

Create, exchange, combine, and use SBOMs

Design principles

- Format independent
- Tool independent
- Organization independent
- Compatible with standards
- Ready for automation

Automotive SBOM

Data fields: the minimum set to identify software

SBOM metadata	
Author Name	**Required**
Timestamp	**Required**
Type	**Required**
Primary Component	**Required**

Two design rules

No dynamic data

Vulnerability status changes daily; keep it outside

No business data

e.g., vehicle model — keep it separate

Component attributes	
Name	**Required**
Version	**Required**
Supplier Name	**Required**
Relationship	**Required**
Unique Identifier	**Required**
File Name	**Required**
Download Location	Optional
Declared License	Optional
Concluded License	**Required**
Cryptographic Hash	Optional
Copyright Notice	**Required**
External Document References	**Required**

Automotive SBOM

A worked example: one component

Field	Value
Component Name	libfoo
Version	1.2.0
Supplier Name	Organization: Foo Project
Unique Identifier	pkg:generic/libfoo@1.2.0 (PURL)
File Name	libfoo-1.2.0.tar.gz
Declared License	**BSD (as written by the author)**
Concluded License	**BSD-3-Clause (as assessed)**
Copyright Notice	Copyright 2026 Foo Project
Relationship	ECU-App CONTAINS libfoo

Why Concluded is required

Declared = the author's claim

Concluded = the assessed result

Downstream users need the result, not the raw claim.

NOASSERTION

Means "could not be determined", not "none". Use only as an exception.

Automotive SBOM

From component SBOMs to a whole-vehicle view

1

Many small SBOMs, not one huge file

Each supplier owns its SBOM; SBOMs are linked by external references

2

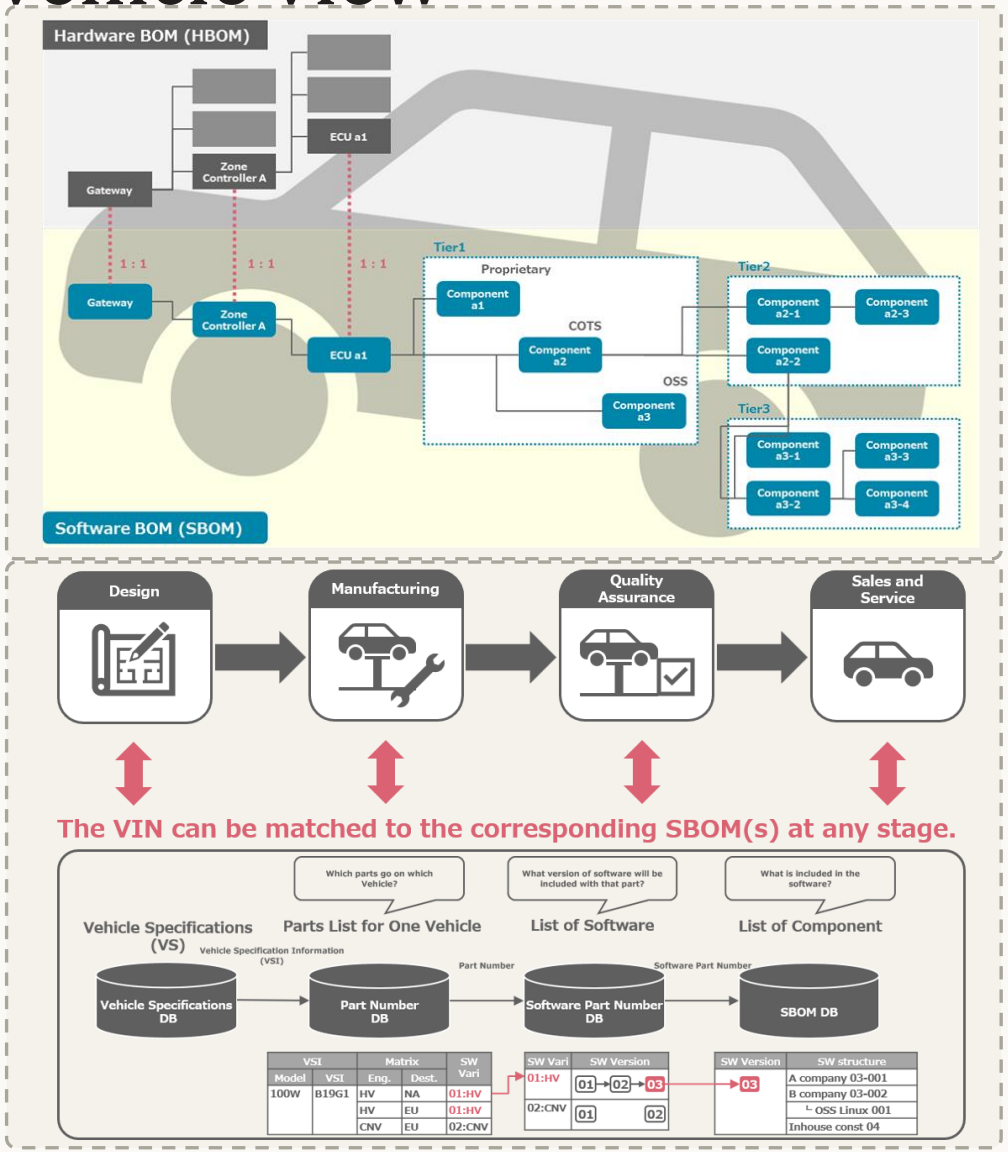
Hardware structure meets software

HBOM / PLM gives the ECU tree; ALM gives the software

3

Traceable after shipment

VIN → software part no. → SBOM
OTA updates make each car unique



Our journey so far

Nov 2025	Action plan at the OpenChain Automotive WG
Dec 2025	Kick-off; first draft published on GitHub
Jan–Feb 2026	Draft moved to Markdown; project restarted
Mar 2026	Japanese-speaking feedback session
Apr, Jun 2026	Updates at the Governance Monthly meeting
Jul–Sep 2026	Issue-by-issue review on GitHub
Oct 2026	**v1.0 published**

64

pull requests merged since July 2026

62 of them in September

About 150 issues and PRs

Open to everyone

Who is building it

Editors

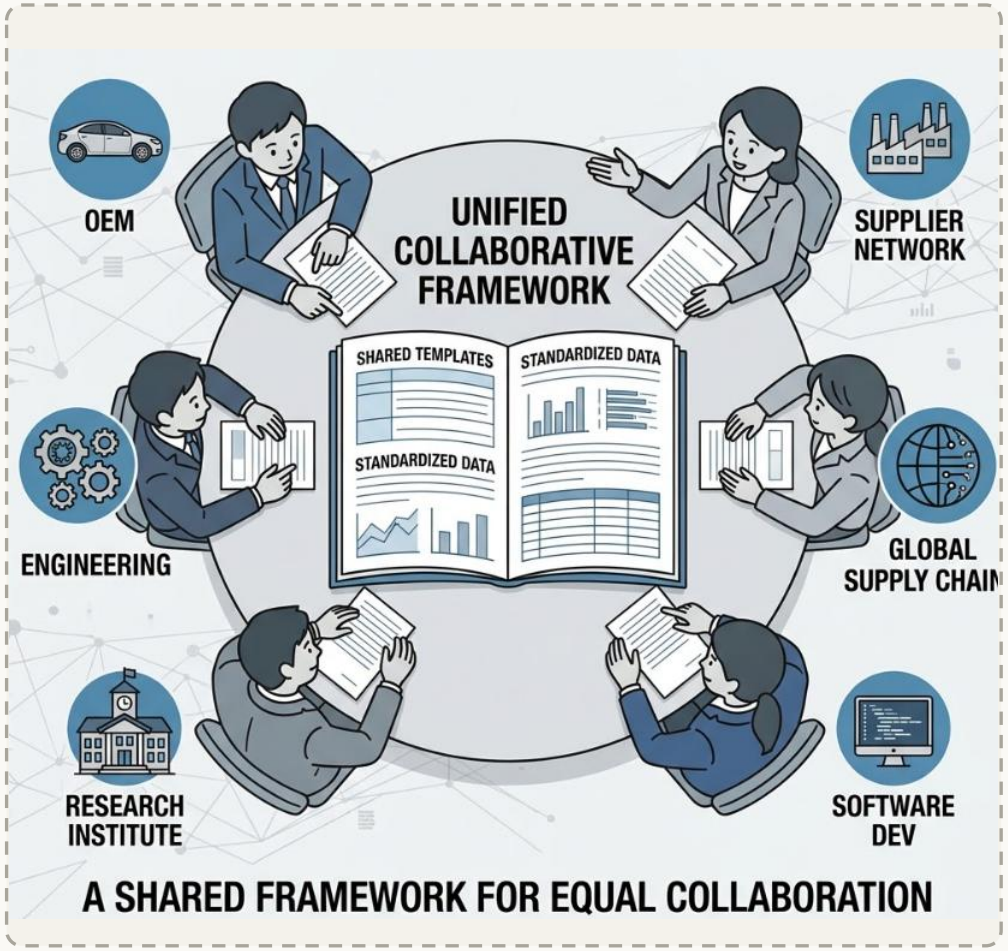
Ayumi Watanabe · Keisuke Takase · Masato Endo

Contributors

Marc-Etienne Vargenau (Nokia) and all reviewers

Home: OpenChain Project, Automotive Work Group under the Linux Foundation

Contributor	Supporter	User
Writes and reviews	Backs the work	Uses it in practice



What we heard, and how v1.0 responds

What we heard	How v1.0 responds	Status
"Recommend SPDX 2.3 and CycloneDX 1.6"	Stay format-neutral; list supported examples	Clarified
"Why is VEX not inside the SBOM?"	New guidance: SBOM identifies, VEX assesses	Clarified
"How does it differ from Catena-X?"	Side-by-side comparison with Catena-X experts	Planned
"Cover commercial and proprietary SW"	Proprietary use case; IDs for non-OSS parts	Reflected
"Make CPE / PURL matching reliable"	Recommend at least one of CPE or PURL	Reflected
"Standardize the process, not only format"	Practice and Process chapter expanded	In progress
"What about AI/ML models and AI coding?"	Scoped as a v2.0 theme	Planned

Channels: Governance Monthly · Japanese-speaking session · GitHub · Google Docs · mailing list

Change 1 — From "Specification" to "Framework"

Aspect	Earlier draft	v1.0 (Sep 2026)
Name	Automotive SBOM Specification	**Automotive SBOM Framework**
Viewpoint	What OEMs require from suppliers	**A common language for the whole chain**
Objectives	3 purposes	**4 objectives, incl. risk management**
Use cases	General SBOM needs	**5 automotive-specific use cases**
Creation	Automation assumed	**Manual, automated, or hybrid**
Formats	Version tips in earlier slides	**Any format that can express the fields**
License	CC0 1.0	**CC BY 4.0, contributors credited**

Automotive SBOM

Change 2 — Clearer rules for data fields

Field	Earlier draft	v1.0
Supplier Name	URL, PURL namespace, or "unknown"	**Legal entity or maintaining project**
Unique Identifier	OSS: PURL or SWHID; COTS: SWID	**OSS: CPE or PURL; non-OSS: any unique ID**
Copyright Notice	Primary component only	**Every component; NOASSERTION if unknown**
Version	Use a hash if there is no version	**Any unique scheme (SemVer, CalVer, hash)**
Timestamp	ISO 8601 as an example	**UTC, YYYY-MM-DDThh:mm:ssZ**
Optional fields	Not defined	**Missing optional ≠ non-conformance**
Type and tool	Free text	**"SBOM Type: Build"; "Tool: name - version"**

CPE, PURL, SWHID: common ways to name a software package so tools can match it.

Automotive SBOM

Change 3 — Formats and alignment with standards

Formats

Any format that can express the required fields

SPDX	**2.x · 3.x**
CycloneDX	**1.4 – 1.7**
Future versions	**Allowed**

vs. 2026 Minimum Elements (CISA et al., Jul 2026)

Covered in v1.0

- Author, timestamp
- Name, version, supplier
- Relationship, identifier
- License, hash
- Generation context (partly)

Out of scope for v1.0

- SBOM author signature
- Tool name and version
- SBOM version
- Format name and version

Also updated: J-Auto-ISAC mapping · EU CRA and METI sections · links to all references

Change 4 — New guidance for daily practice

CVE published
A new vulnerability is announced

→

SBOM
Which ECUs contain the component?

→

VEX
Is the product really affected?

→

Fix
OTA update, plus a safety check

VEX = a statement of whether a product is affected by a known vulnerability

Vulnerability management (§3.4.5)

- SBOM = static: changes per release
- VEX = dynamic: changes daily
- Link by identifiers; do not embed

Concluded license (§3.4.6)

- Tools suggest; people may decide
- Manual, automated, or hybrid
- Proprietary use case also added

Toward v2.0: five themes

1	**Vulnerability and VEX**	Point to components without ambiguity
2	**Functional safety**	Link security fixes to safety impact
3	**AI-generated code**	OSS snippets slipping into code
4	**AIBOM**	AI models and datasets as components
5	**Proprietary and COTS**	Transparency without exposing IP

Continuing in v1.x

- Practice sections: request, generate, operate SBOMs
- Comparison with Catena-X
- 2026 Minimum Elements items (e.g., signature, tool)

v2.0 focus 1 — Vulnerabilities, VEX, and safety

Vulnerability and VEX

- How should VEX point to a component?
- Which format? (e.g., CSAF VEX)
- Keep separate, or link from the SBOM?
- Who issues VEX across tiers?

Functional safety

- A security patch is also a safety change
- Use SBOM relationships for impact analysis
- Connect ISO/SAE 21434 and ISO 26262
- Keep SBOM and VEX alive for 10+ years

Open question: Where should vulnerability status live over a 15-year vehicle life?

v2.0 focus 2 — Code written with generative AI

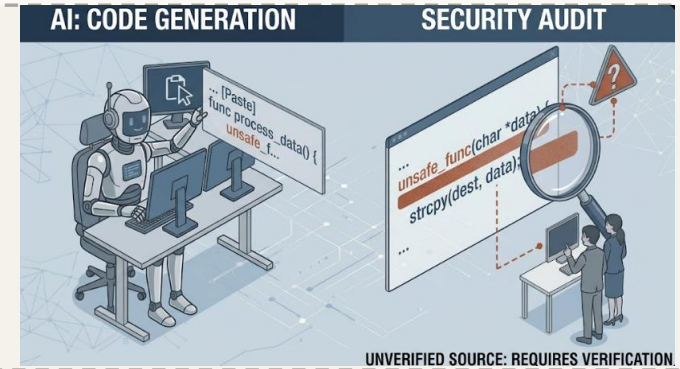
The problem
AI coding agents can copy OSS snippets without origin or license information.

Why it is hard to detect

- Dependency-based SCA misses snippets
- Snippet matching is less precise
- Forks blur the version
- No tool can promise 100% detection

Candidate practices

- SCA plus snippet scanning
- Prompt and skill rules: do not copy OSS
- Record where AI-assisted code is used



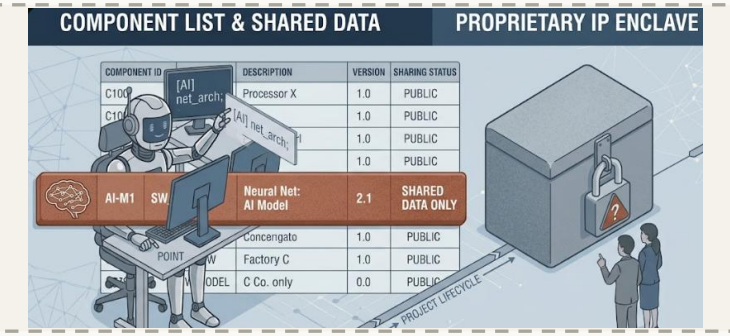
v2.0 focus 3 — AIBOM and proprietary software

AIBOM: AI models as components

- Name, version, training data
- License and intended use
- Base: SPDX 3.0 AI / Dataset profiles
- Base: CycloneDX ML-BOM

Proprietary and COTS software

- Most vehicle software is not OSS
- Identifiers when no PURL / CPE exists
- Right level of detail; sharing under NDA



Open question: How much detail is enough for proprietary parts without exposing IP?

SUMMARY

Roadmap and how to take part

Oct 2026
v1.0 published

Oct 7–9, 2026
Open-source Summit EU

Dec 7–9, 2026
Open-source Summit Japan

2027
v2.0 drafting (proposed)

Take part

GitHub: github.com/OpenChain-Project/Automotive-SBOM

Mailing list: lists.openchainproject.org/g/automotive-sbom

Meetings: Automotive OSS Governance Monthly

Bring an issue, a PR, or an agenda item.

Thank you

Questions and comments are welcome.